ACTUAL

Dossier professionnel

Développeur web et web mobile

Information	Valeur
Candidat	Christophe Barrere
Titre professionnel visé	Développeur web et web mobile
Référence	RNCP 37674 Niveau 5
Modalité d'accès	Parcours de formation À compléter si nécessaire
Organisme de formation	À compléter
Session d'examen	À compléter
Projet présenté	Actual Application web de réservation de cours de sport

Version préparatoire : ce document rassemble les textes à reprendre dans le modèle officiel du dossier professionnel fourni par l'organisme de formation ou le centre d'examen.

Sommaire

1 Présentation du candidat et du projet

2 Synthèse des compétences

3 Activité type 1 Développer la partie front end

4 Activité type 2 Développer la partie back end

5 Bilan professionnel

6 Liste des annexes et des preuves

7 Déclaration sur l'honneur

Informations à compléter

- nom et adresse de l'organisme de formation
- dates exactes de la formation et de la session d'examen
- nom du formateur ou du tuteur si cette information est demandée
- captures d'écran définitives et résultats réels des tests
- URL de production lorsque l'application sera déployée
- dates de réalisation de chaque exemple

1 Présentation du candidat et du projet

1.1 Mon parcours

Je suis en reconversion professionnelle vers le développement web. Ma formation m'a permis d'apprendre à analyser un besoin, organiser un projet, développer des interfaces et créer la partie serveur d'une application. J'ai choisi de réaliser Actual afin de mettre en pratique ces compétences dans un projet complet et facile à démontrer.

Actual est une application web de réservation de cours de sport. Un visiteur consulte le planning et le détail des cours. Après avoir créé un compte, il peut réserver une place, consulter son historique et annuler une réservation. Un administrateur gère les cours, les réservations et les utilisateurs depuis l'administration Django.

1.2 Conditions de réalisation

Élément	Description
Nature du travail	Projet de formation réalisé individuellement
Environnement	Windows 11, WSL Ubuntu et Visual Studio Code
Langage et framework	Python 3 et Django
Interface	HTML, CSS, Bootstrap et Bootswatch Simplex
Base de développement	SQLite
Versionnement	Git et dépôt GitHub
Dépôt	https://github.com/ChristopheBarrere/actual_rncp

1.3 Besoin auquel répond Actual

Une salle de sport doit publier son planning et connaître le nombre de participants à chaque cours. Lorsque les inscriptions sont prises par plusieurs moyens, le personnel peut perdre du temps et dépasser la capacité prévue. Actual centralise les cours et permet aux adhérents de gérer leurs réservations.

1.4 Périmètre

- création de compte, connexion et déconnexion
- consultation du planning et du détail d'un cours
- calcul du nombre de places disponibles
- réservation et annulation
- historique personnel des réservations
- gestion des cours, réservations et utilisateurs dans l'administration

Le paiement, les abonnements et la messagerie ne font pas partie de la version présentée.

2 Synthèse des compétences

Compétence	Mise en œuvre dans Actual	État

Compétence	Mise en œuvre dans Actual	État
Installer et configurer l'environnement	WSL, environnement virtuel, Django, Git et VS Code	Réalisé
Maquetter les interfaces	Définition des pages, de la navigation et de l'identité Actual	Preuves à joindre
Réaliser des interfaces statiques	Templates HTML, CSS, Bootstrap et Simplex	Réalisé
Développer la partie dynamique	Templates conditionnels, formulaires et messages Django	Partiel côté client
Mettre en place une base relationnelle	Modèles User, Course et Reservation	Réalisé avec SQLite
Développer l'accès aux données	ORM Django, filtres, relations et select_related	Réalisé
Développer les composants métier	Inscription, capacité, réservation, annulation et administration	Réalisé
Documenter le déploiement	Procédure PostgreSQL, variables d'environnement et hébergement	Preuve finale à compléter

Point de vigilance : la version actuelle ne contient pas de fonctionnalité JavaScript personnelle. Le rendu dynamique repose principalement sur Django et ses templates. Je présente ce point avec transparence et je peux renforcer cette compétence par un filtre de planning en JavaScript si le calendrier le permet.

3 Activité type 1 Développer la partie front end

Cette activité regroupe la préparation de l'environnement, l'organisation des interfaces et la réalisation des pages visibles par l'utilisateur. J'ai cherché à obtenir une navigation simple et une présentation cohérente sur toutes les pages.

Exemple 1 Installer et organiser l'environnement de travail

Compétences principalement illustrées : Installer et configurer son environnement de travail

1 Tâches et opérations réalisées

1. J'ai créé le dossier du projet dans WSL et ouvert ce dossier avec Visual Studio Code.
2. J'ai créé un environnement virtuel Python afin d'isoler les dépendances.
3. J'ai installé Django avec pip et enregistré les dépendances dans requirements.txt.
4. J'ai créé le projet actual et l'application courses.
5. J'ai configuré l'application, les templates et les fichiers statiques dans settings.py.
6. J'ai initialisé le dépôt Git puis publié le projet sur GitHub.

2 Moyens utilisés

Moyen	Utilisation
WSL Ubuntu	Exécuter Python, Django, Git et les commandes du projet
Visual Studio Code	Écrire et organiser le code depuis WSL
venv et pip	Isoler et installer les paquets Python
Git et GitHub	Versionner et sauvegarder le projet
Documentation Django	Vérifier la configuration et les commandes

3 Résultat obtenu

L'environnement est reproductible à partir du dépôt et du fichier requirements.txt. Le serveur de développement démarre avec python manage.py runserver. Les migrations créent la structure de la base sur une nouvelle installation.

4 Organisation du travail

J'ai réalisé cet exemple individuellement dans le cadre de mon projet de formation. J'ai découpé le travail en étapes courtes et créé des commits en anglais après chaque fonction importante. Les éventuels échanges avec le formateur sont à préciser ici : À compléter.

Preuves à joindre

- capture du terminal WSL avec l'environnement virtuel actif
- arborescence du projet dans Visual Studio Code
- fichier requirements.txt
- historique des premiers commits GitHub

Exemple 2 Réaliser une interface responsive avec le thème Actual

Compétences principalement illustrées : Maquetter et réaliser des interfaces utilisateur statiques

1 Tâches et opérations réalisées

J'ai commencé par identifier les pages nécessaires : accueil, planning, détail d'un cours, inscription, connexion et historique des réservations. J'ai ensuite défini une navigation commune et une identité visuelle adaptée à une salle de sport.

1. J'ai créé le template base.html pour regrouper la navigation et les ressources communes.
2. J'ai intégré Bootstrap et le thème Bootswatch Simplex.
3. J'ai ajouté le logo Actual, une image de salle de sport et un fichier CSS personnalisé.
4. J'ai adapté les templates du planning, du détail, de la connexion, de l'inscription et des réservations.
5. J'ai utilisé la grille et les composants Bootstrap pour adapter l'affichage aux différentes largeurs d'écran.

2 Choix réalisés

- une image principale assombrie afin de conserver la lisibilité du texte
- une couleur rouge pour rappeler l'identité Actual
- des boutons et des liens nommés explicitement
- un menu différent selon que l'utilisateur est connecté ou non
- un template commun afin d'éviter la répétition du code HTML

3 Résultat obtenu

Les pages utilisent le même thème et restent lisibles sur ordinateur et sur téléphone. L'héritage des templates facilite la maintenance : une modification de la navigation dans base.html s'applique à l'ensemble du site.

4 Difficultés et solutions

La principale difficulté a été de conserver la lisibilité au-dessus de l'image de fond. J'ai ajouté un assombrissement et ajusté les couleurs du texte. J'ai également corrigé la position des éléments du menu afin que le nom de l'utilisateur ne soit pas placé entre deux actions principales.

5 Organisation du travail

J'ai réalisé les pages et leur intégration. Le thème Bootswatch fournit une base graphique, mais j'ai adapté la structure des templates et le CSS à l'application Actual.

Preuves à joindre

- maquettes ou croquis des pages à ajouter
- captures de l'accueil, du planning et du détail
- capture en affichage mobile
- extraits de base.html et du fichier CSS

Exemple 3 Adapter l'interface à l'utilisateur connecté

Compétences principalement illustrées : Développer la partie dynamique des interfaces utilisateur

1 Tâches et opérations réalisées

J'ai développé une interface dont le contenu change selon l'état de connexion, l'état du cours et l'existence d'une réservation.

- affichage des boutons de connexion et d'inscription pour un visiteur
- affichage de Mes réservations et de la déconnexion pour un membre
- bouton Réserver uniquement lorsque le cours est disponible
- bouton Annuler lorsque l'utilisateur possède déjà une réservation confirmée
- message spécifique pour un cours complet ou annulé
- messages de confirmation ou d'erreur après une action

2 Moyens utilisés

J'ai utilisé le langage de templates Django, notamment les instructions if, elif, else et for. Les vues transmettent au template le cours, l'utilisateur et l'information has_confirmed_reservation. Les formulaires utilisent la méthode POST et le jeton CSRF.

3 Résultat obtenu

L'utilisateur ne voit que les actions compatibles avec sa situation. Cette adaptation limite les erreurs et rend le parcours plus facile à comprendre.

4 Limite identifiée

La version présentée utilise principalement un rendu dynamique côté serveur. Elle ne comporte pas encore de fonctionnalité JavaScript personnalisée. Cette limite est déclarée afin de ne pas présenter une compétence qui n'aurait pas été réellement mise en œuvre.

Preuves à joindre

- captures du détail avant et après connexion
- capture d'un cours complet ou annulé
- extrait des conditions du template course_detail.html
- extrait de l'affichage des messages Django

4 Activité type 2 Développer la partie back end

Cette activité concerne la structure des données, leur accès avec l'ORM, les règles de réservation, la sécurité côté serveur, les tests et la préparation du déploiement.

Exemple 1 Concevoir et mettre en place la base relationnelle

Compétences principalement illustrées : Mettre en place une base relationnelle et développer l'accès aux données

1 Tâches et opérations réalisées

J'ai identifié trois entités : User, Course et Reservation. Django fournit le modèle User. J'ai créé les modèles Course et Reservation dans courses/models.py.

Modèle	Données principales	Rôle
User	username, email, password et autorisations	Compte et authentification
Course	titre, coach, date, durée, capacité et état	Cours proposé par la salle
Reservation	utilisateur, cours, statut et dates	Association entre un membre et un cours

J'ai relié Reservation à User et Course avec deux clés étrangères. J'ai ajouté une contrainte d'unicité sur le couple user et course. Cette contrainte protège la cohérence de la base même si un contrôle applicatif est oublié.

2 Accès aux données

- all et order_by pour afficher les cours
- get_object_or_404 pour récupérer un cours ou renvoyer une erreur 404
- filter et first pour rechercher une réservation existante
- create et save pour enregistrer les changements
- select_related pour charger les cours associés à l'historique

3 Calcul de la disponibilité

Je ne stocke pas le nombre de places restantes. Le modèle le calcule à partir de la capacité et du nombre de réservations confirmées. Ce choix évite de maintenir deux valeurs pouvant devenir incohérentes.

4 Résultat obtenu

Les migrations permettent de créer la même structure de base sur une nouvelle installation. L'ORM permet de manipuler les données avec des objets Python tout en générant les requêtes SQL nécessaires.

Preuves à joindre

- schéma relationnel User, Reservation et Course
- extrait de courses/models.py
- migration initiale
- capture des tables ou de l'administration Django

Exemple 2 Développer et sécuriser la réservation

Compétences principalement illustrées : Développer des composants métier côté serveur

1 Tâches et opérations réalisées

J'ai développé les vues de réservation et d'annulation dans courses/views.py. Elles appliquent les règles métier avant de modifier la base.

1. Vérifier que l'utilisateur est connecté avec login_required.
2. Accepter uniquement une requête POST avec require_POST.
3. Récupérer le cours demandé.
4. Refuser la réservation si le cours est annulé.
5. Refuser la réservation si le cours est complet.
6. Refuser une seconde réservation confirmée.
7. Réactiver une réservation annulée ou créer une nouvelle réservation.
8. Afficher un message et rediriger vers le détail du cours.

2 Annulation

Pour annuler, je recherche uniquement une réservation confirmée appartenant à request.user. Je remplace son statut par cancelled au lieu de supprimer la ligne. L'historique reste disponible et la place est immédiatement libérée.

3 Sécurité appliquée

Risque	Protection
Action envoyée depuis un autre site	Jeton CSRF
Accès sans connexion	Décorateur login_required
Modification par une URL GET	Décorateur require_POST
Annulation d'une autre personne	Filtre user=request.user
Double réservation	Contrôle dans la vue et contrainte en base
Injection SQL	Requêtes construites avec l'ORM Django

4 Résultat obtenu

Le parcours fonctionne de l'inscription à l'annulation. Les règles sont appliquées côté serveur, même si un utilisateur tente de modifier directement l'URL ou le formulaire.

5 Amélioration envisagée

Pour une application très fréquentée, j'ajouterais une transaction et un verrouillage lors de la réservation de la dernière place. Cette amélioration limiterait le risque de deux réservations simultanées.

Preuves à joindre

- extraits des vues reserve_course et cancel_reservation
- capture d'une réservation confirmée
- capture de l'historique après annulation

- capture d'un message de refus sur un cours complet

Exemple 3 Administrer, tester et préparer la livraison

Compétences principalement illustrées : Développer des composants serveur et documenter le déploiement

1 Administration

J'ai enregistré Course et Reservation dans l'administration Django. J'ai ajouté des colonnes, des filtres, une recherche et un tri afin de faciliter la gestion. Les utilisateurs sont administrés avec le module d'authentification fourni par Django.

2 Tests

J'ai écrit quatre tests automatisés avec django.test.TestCase. Ils vérifient l'affichage de la liste, l'affichage du détail, la réservation par un membre connecté et l'annulation.

Test	Résultat attendu
Planning	La réponse affiche le cours créé pour le test
Détail	La réponse affiche les informations du cours demandé
Réservation	Une réservation confirmée est créée
Annulation	Le statut devient cancelled

3 Préparation du déploiement

J'ai documenté le passage prévu de SQLite à PostgreSQL, l'utilisation de variables d'environnement, la désactivation de DEBUG, la configuration des hôtes autorisés, les migrations et la collecte des fichiers statiques.

Au moment de la rédaction, PostgreSQL et l'URL publique restent à valider. Je ne les présente donc pas comme des réalisations terminées.

4 Résultat et recul

L'administration rend l'application exploitable sans développer un espace de gestion supplémentaire. Les tests couvrent le parcours principal. Pour améliorer la qualité, je compléterais la couverture avec les cas de cours complet, de cours annulé et d'accès non autorisé.

Preuves à joindre

- captures de l'administration des cours et des réservations
- fichier courses/tests.py
- capture réelle de python manage.py test
- capture réelle de python manage.py check
- procédure PostgreSQL et URL publique après déploiement

5 Bilan professionnel

5.1 Compétences acquises

La réalisation d'Actual m'a permis de comprendre le circuit complet d'une application web : une URL reçoit la requête, une vue applique les règles métier, l'ORM accède aux données et un template génère la réponse HTML.

- préparer un environnement Python isolé et reproductible
- organiser un projet Django selon l'architecture MVT
- concevoir une base relationnelle avec des clés étrangères et une contrainte
- développer un parcours authentifié et des règles métier côté serveur
- protéger les formulaires et contrôler les droits d'accès
- créer une interface responsive commune à plusieurs pages
- écrire des tests automatisés et versionner le travail avec Git

5.2 Difficultés rencontrées

Difficulté	Solution retenue
Calculer les places disponibles	Calculer la valeur depuis les réservations confirmées
Empêcher un doublon	Contrôle dans la vue et contrainte d'unicité
Conserver un thème identique	Utiliser un template base et une feuille CSS commune
Distinguer les droits	Authentification utilisateur et administration Django
Conserver l'historique	Changer le statut au lieu de supprimer la réservation

5.3 Axes de progression

- valider PostgreSQL et le déploiement HTTPS
- ajouter une fonctionnalité JavaScript personnelle
- augmenter la couverture des tests
- réaliser un audit d'accessibilité plus complet
- mettre en place une intégration continue
- ajouter une transaction pour les réservations simultanées

5.4 Apport du projet

Ce projet m'a appris à relier les différentes parties d'une application et à justifier mes choix. Il m'a également montré l'importance de limiter le périmètre, de tester progressivement et de documenter ce qui reste à faire.

6 Liste des annexes et des preuves

Annexe	Preuve	État
A1	Schéma de l'architecture MVT	Disponible
A2	Modèle relationnel User, Reservation et Course	Disponible
A3	Capture de la page d'accueil	À insérer
A4	Capture du planning et du détail	À insérer
A5	Capture de Mes réservations	À insérer
A6	Capture de l'administration Django	À insérer
A7	Extrait des modèles	À sélectionner
A8	Extrait des vues de réservation et d'annulation	À sélectionner
A9	Résultat des tests automatisés	À exécuter et capturer
A10	Historique Git et dépôt GitHub	Disponible
A11	URL de production	À ajouter après déploiement

7 Déclaration sur l'honneur

Je soussigné Christophe Barrere déclare sur l'honneur que les informations fournies dans ce dossier sont exactes et que les réalisations présentées correspondent à des travaux auxquels j'ai personnellement contribué.

Cette déclaration doit être comparée au texte demandé dans le modèle officiel remis par l'organisme de formation ou le centre d'examen.

Fait à : À compléter

Le : À compléter

Signature :